Charte d'équipe CyberSentinel

Rédigée en séance 2 (12 novembre 2026), animée par Elias. Chaque membre la relit et la valide (tableau en bas de page).

1. Rôles

Membre	Filière	Rôle principal	Binôme de secours
Elias Perez	Data/IA	Porteur de projet : pilotage, Kanban, étiquettes du classifieur	Nolan
Imad Belghazy	Data/IA	Co-porteur : état de l'art, Presidio, métriques	Aloïs
Nolan	Data/IA	Jeu de prompts, interface `detect()` entre le proxy et la détection	Elias
Arno	Cyber	Proxy mitmproxy, certificat, procédure d'installation	Kiliann
Aloïs	Cyber	Règles M1 (données sensibles), note RGPD	Imad
Kiliann	Cyber	Module 2 (paquets inventés par les IA), règles M2	Arno
Yarkin	Dev	Dépôt, infra Docker, API FastAPI, architecture	Fati
Fati	Dev	Tableau de bord : maquettes et front React	Yarkin

Binôme de secours : si quelqu'un est absent, son binôme reprend ses tâches prioritaires (★).

2. Communication

- **Canal principal** : *à compléter (Discord / WhatsApp)*. Toutes les questions passent par là, pas en message privé, pour que tout le monde suive.
- **Temps de réponse** : on répond sous 24 h en semaine.
- **Blocage** : si on est bloqué depuis plus de 30 minutes en séance (ou une demi-journée hors séance), on le dit dans le canal et on ajoute l'étiquette `bloquant` à sa carte.
- **Décisions** : les décisions qui touchent plusieurs filières (format d'événement, architecture, choix d'outils) se prennent au point de midi ou de fin de séance et sont notées dans le compte rendu (`docs/comptes-rendus/`).

3. Horaires et rituels

- **Horaires de séance** : *à compléter*.
- **Début de séance (première heure si besoin)** : on se met d'accord sur l'objectif du jour.
- **Point de midi (10 min)** : chacun montre où il en est et on réaffecte ce qui bloque.
- **Point de fin (20 min)** : chacun montre son livrable et le pousse sur GitHub.
- **Retard ou absence** : on prévient dans le canal **avant** le début de la séance, et son binôme est au courant.

4. Règles Git

Branches

Branche	Rôle
`main`	Uniquement du code qui marche. Personne ne pousse directement dessus.
`cyber`, `data`, `dev`	Branches d'intégration de chaque filière.
`feature/<sujet>`, `fix/<sujet>`, `docs/<sujet>`	**Une branche par tâche**, créée à partir de la branche de sa filière.

> À valider en équipe : soit les branches de filière servent d'intégration (proposition ci-dessus), soit on les supprime et toutes les branches de tâche partent de `main` (voir `CONTRIBUTING.md`).

Commits

Format : `type: ce qui a été fait`, en français. Les types : `feat`, `fix`, `docs`, `test`, `refactor`, `chore`.

Pull requests

1. On pousse sa branche et on ouvre une pull request.
2. On lie la carte du Kanban (`Closes #12`).
3. **Quelqu'un d'autre relit et valide** avant la fusion (de préférence son binôme).
4. On ne fusionne pas sa propre pull request sans relecture.

Interdits

- Pas de secrets dans le dépôt : mots de passe, clés API, certificats. Ils vont dans `.env`, qui est ignoré par Git.
- Pas de vraies données personnelles, uniquement des données inventées.
- Pas de `git push --force` sur `main` ni sur les branches de filière.

5. Kanban

Il est sur GitHub Projects, avec une carte par tâche.

- On prend une carte → **En cours**, et on s'assigne.
- On ouvre une pull request → **En relecture**.
- C'est fusionné → **Terminé**.

6. Règle d'or du projet

On ne lance les modules bonus (3 et 4) que si les modules 1 et 2 marchent vers la séance 7. **Mieux vaut 2 modules solides que 4 bancals.**

7. Validation

Membre	Lu et validé
Elias	☐
Imad	☐
Nolan	☐
Arno	☐
Aloïs	☐
Kiliann	☐
Yarkin	☐
Fati	☐